

K-디지털 트레이닝

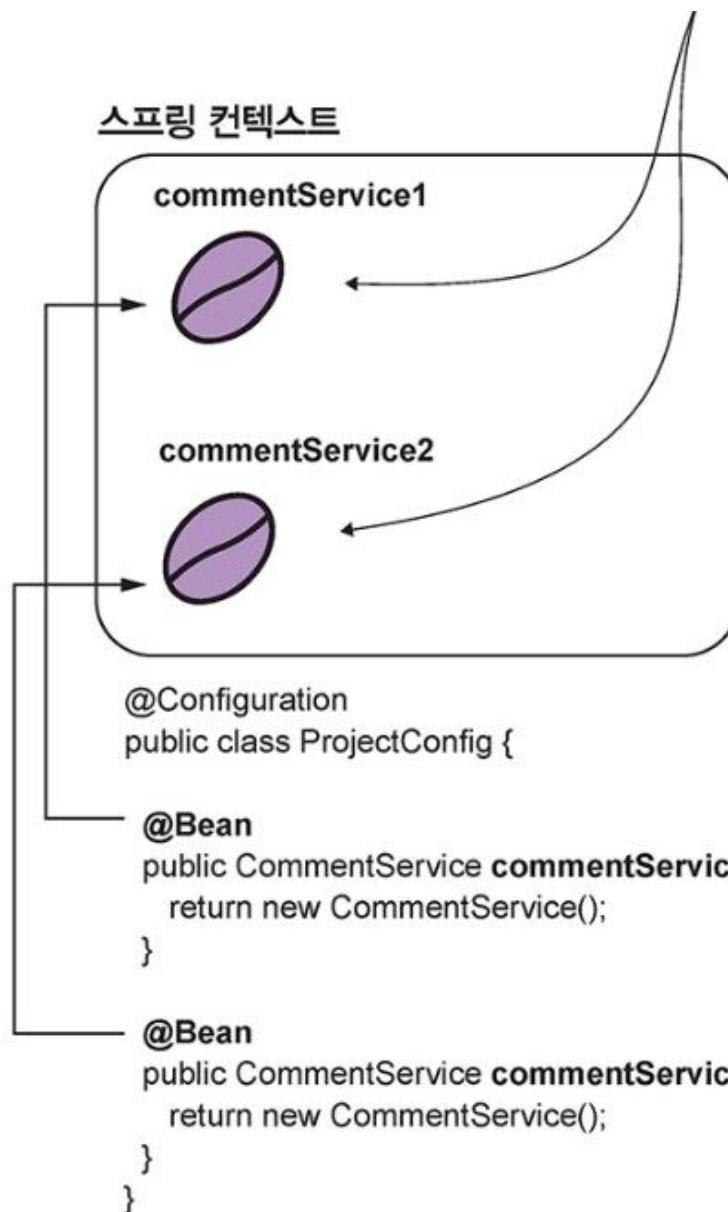
스프링 컨텍스트- 빈의 스코프 및 수명 주기

[KB] IT's Your Life

✓ 싱글톤 빈의 작동 방식

○ 스프링에서 싱글톤 스코프

스프링에서는 구성 클래스에서 @Bean 애너테이션이 달린 메서드로 동일한 타입의 빈을 최대한 많이 정의할 수 있다. 이 각각의 빈을 싱글톤이라고 한다.



스프링에서 싱글톤 스코프

하지만 스프링에서 싱글톤은 고유한 이름에 대해 동일한 인스턴스를 의미한다. 스프링의 컨텍스트에서 인스턴스 이름이 다르다면 동일한 클래스의 인스턴스라도 여러 개 가질 수 있다.

✓ @Bean으로 싱글톤 스코프의 빈 선언하기

```
@Configuration
public class ProjectConfig {

    @Bean
    public CommentService commentService() {
        return new CommentService();
    }
}
```

빈을 추가한다.

```
public class Main {

    public static void main(String[] args) {
        var c = new AnnotationConfigApplicationContext(ProjectConfig.class);

        var cs1 = c.getBean("commentService", CommentService.class);
        var cs2 = c.getBean("commentService", CommentService.class);

        boolean b1 = cs1 == cs2;

        System.out.println(b1);
    }
}
```

컨텍스트를 생성한다.

스프링 컨텍스트

commentService

빈을
얻는다.

빈을
얻는다.

이 두 줄의 코드에서 컨텍스트에 있는 동일한 빈을 참조한다.
cs1과 cs2 변수는 동일한 객체 인스턴스의
참조를 갖기 때문에 b1 값은 true가 된다.

```
public class CommentService { }
```

```
@Configuration
```

```
public class ProjectConfig {
```

```
    @Bean ← 스프링 컨텍스트에 CommentService 빈을 추가한다.
```

```
    public CommentService commentService() {
```

```
        return new CommentService();
```

```
    }
```

```
}
```

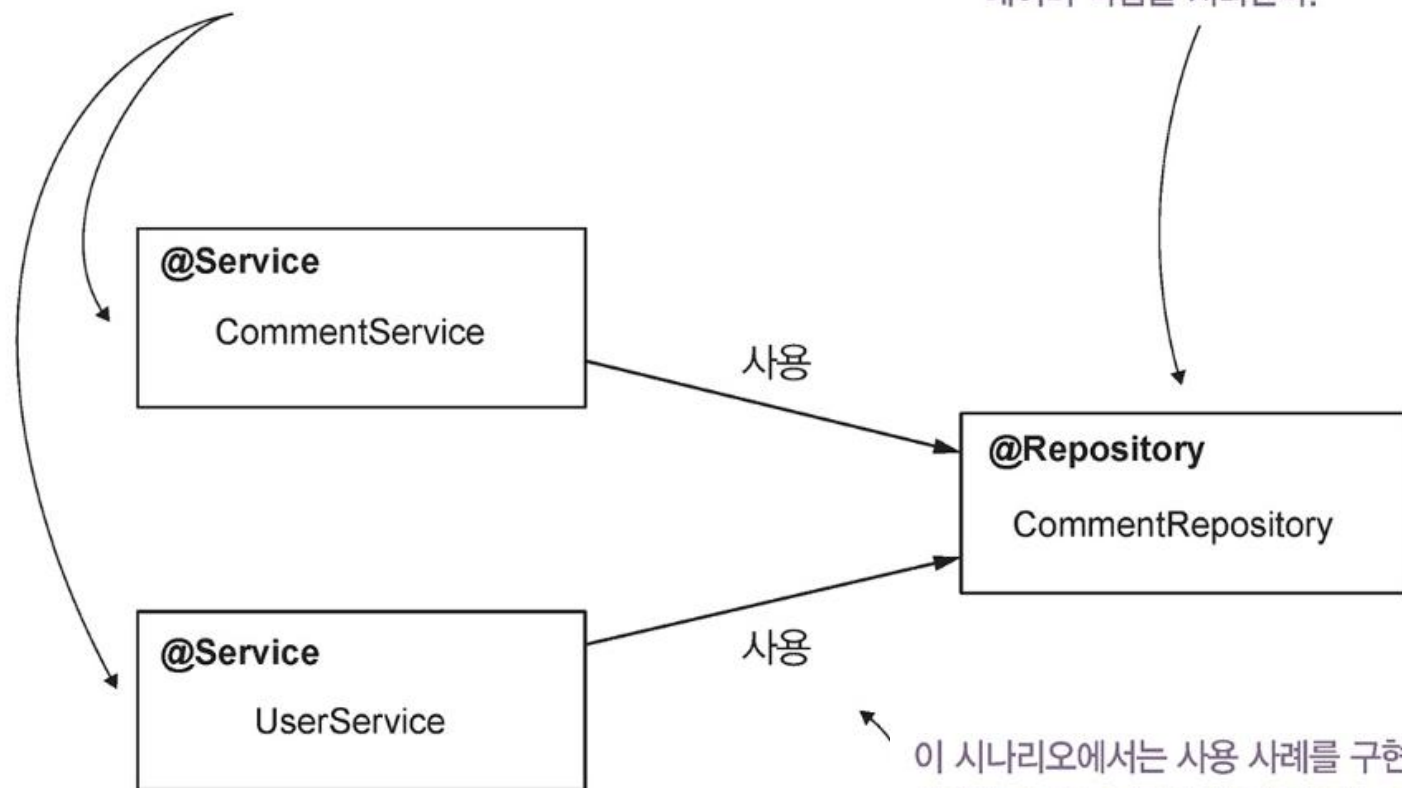
✓ 싱글톤 빈에 대한 스프링 동작을 테스트하는 데 사용된 Main 클래스

```
public class Main (  
    public static void main(String[] args) {  
        var c = new AnnotationConfigApplicationContext(ProjectConfig0.class);  
        var cs1 = c.getBean("commentService", CommentService.class);  
        var cs2 = c.getBean("commentService", CommentService.class);  
  
        boolean b1 = cs1 == cs2; ← 두 변수는 동일한 참조 값을 갖기 때문에 비교 연산 결과는 항상 참이다.  
        System.out.println(b1);  
    }  
}
```

✓ 스테레오타입 애너테이션으로 싱글톤 빈 선언하기

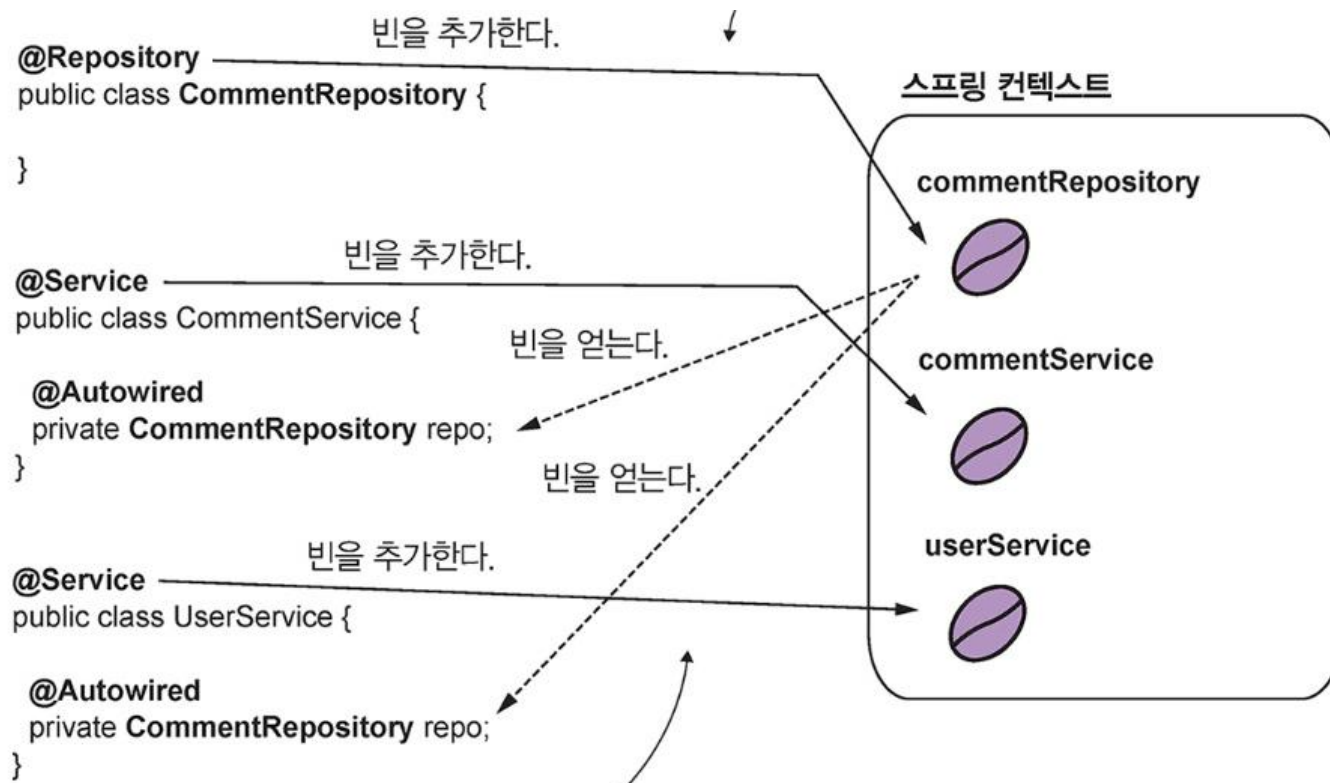
일반적인 클래스 설계에서 서비스 클래스는
사용 사례(use case)를 구현한다.

리포지터리 클래스는 지속성이 있는
데이터 작업을 처리한다.



이 시나리오에서는 사용 사례를 구현하려고 두 서비스
클래스가 리포지터리에 의존한다고 가정한다.

@Repository 스테레오타입 애너테이션을 사용하면 스프링은 컨텍스트에 CommentRepository 타입의 빈을 컨텍스트에 추가한다. 기본적으로 빈은 싱글톤이므로 스프링은 하나의 인스턴스를 생성하고 이름을 부여한다. 스테레오타입 애너테이션을 사용할 때는 클래스 이름이 빈 이름이 된다.



두 서비스 클래스 모두 @Autowired를 사용하여 스프링에 CommentRepository 타입의 빈을 주입하도록 요청한다. 둘 모두 스프링은 스프링 컨텍스트에 있는 해당 빈 인스턴스에 대한 참조만 주입한다. 이제 두 서비스 모두 동일한 CommentRepository 인스턴스를 참조한다.

```
@Repository
public class CommentRepository { }
```

@Service

```
public class CommentService {
    @Autowired
    private CommentRepository commentRepository;

    public CommentRepository getCommentRepository() {
        return commentRepository;
    }
}
```

@Service

```
public class UserService {
    @Autowired
    private CommentRepository commentRepository;

    public CommentRepository getCommentRepository() {
        return commentRepository;
    }
}
```

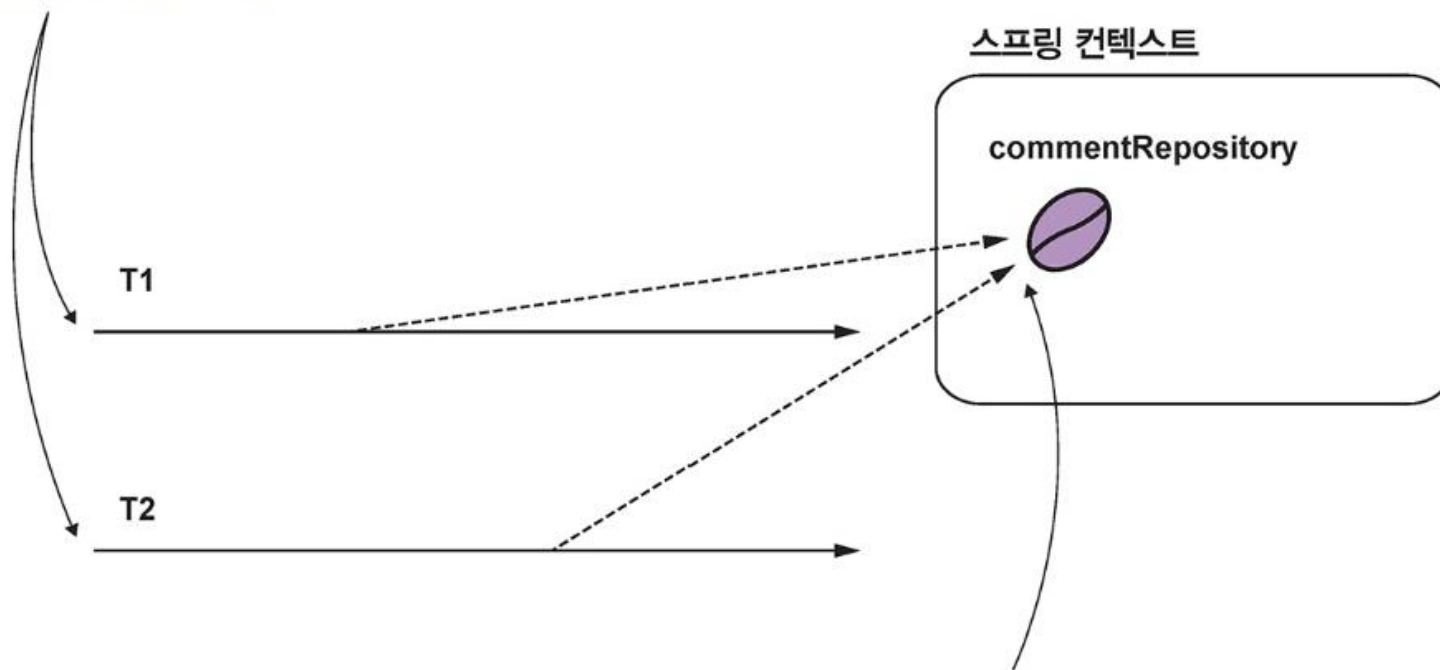
```
@Configuration
@ComponentScan(basePackages= { "services", "repositories"})
public class ProjectConfig {

}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        var c = new AnnotationConfigApplicationContext(ProjectConfig.class);  
  
        var s1 = c.getBean(CommentService.class);  
        var s2 = c.getBean(UserService.class);  
  
        boolean b = s1.getCommentRepository() == s2.getCommentRepository();  
        → CommentRepository 의존성이 싱글톤이므로 두 서비스는 동일한 참조를 사용한다. 결과는 true  
  
        System.out.println(b);  
    }  
}
```

✓ 실제 시나리오의 싱글톤 빈

이 화살표들은 T1과 T2라는 서로 다른 스레드의
실행 타임라인을 나타낸다.



두 스레드가 동일한 인스턴스에 동시에 액세스하고
변경하려고 하면 경쟁 상태가 발생한다.

✓ 빈의 사용 방법

- 스프링 컨텍스트에서 객체 빈을 만드는 경우, 불변인 경우에만 싱글톤으로 만든다.

```
@Service
public class CommentService {

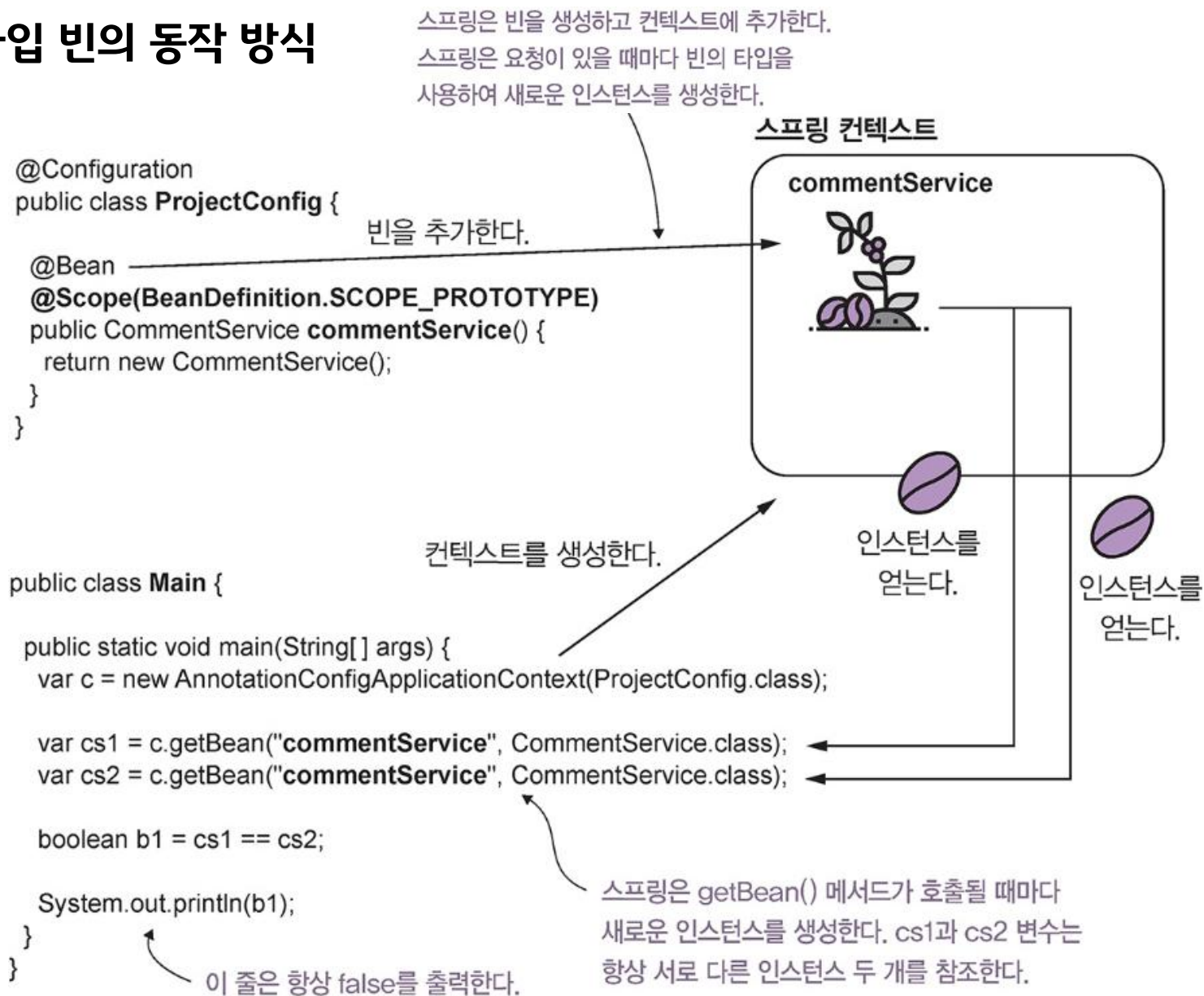
    private final CommentRepository commentRepository;

    public CommentService(CommentRepository commentRepository) {
        this.commentRepository = commentRepository;
    }

    public CommentRepository getCommentRepository() {
        return commentRepository;
    }
}
```

- ✓ 즉시 및 지연 인스턴스 생성 방식

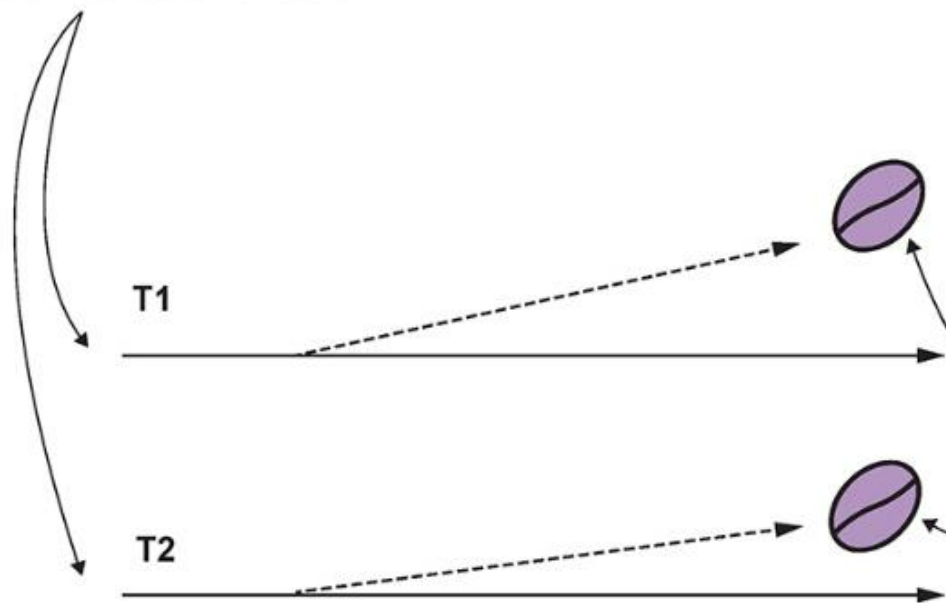
✓ 프로토타입 빈의 동작 방식



✓ 여러 스레드가 특정 프로토타입 빈을 요청하는 경우

- 각각의 스레드는 서로 다른 인스턴스를 얻음 → 경쟁 상태에 빠지지 않는다.

이 화살은 T1과 T2라는 서로 다른 스레드의 실행 타임라인을 나타낸다.



스프링 컨텍스트

commentRepository

두 스레드가 이 빈을 얻는다면 각각의 스레드는 서로 다른 객체 인스턴스를 얻을 것이다.
즉, 각 스레드는 독립적인 인스턴스를 갖는다.

✓ @Bean으로 프로토타입 스코프의 빈 선언하기

```
public class CommentService {  
  
}
```

@Configuration

```
public class ProjectConfig {
```

```
    @Bean
```

```
    @Scope(BeanDefinition.SCOPE_PROTOTYPE) ← 이 빈을 프로토타입 스코프로 만든다.
```

```
    public CommentService commentService() {
```

```
        return new CommentService();
```

```
    }
```

```
}
```

✓ @Bean으로 프로토타입 스코프의 빈 선언하기

```
public class Main {  
  
    public static void main(String[] args) {  
        var c = new AnnotationConfigApplicationContext(ProjectConfig.class);  
  
        var cs1 = c.getBean("commentService", CommentService.class);  
        var cs2 = c.getBean("commentService", CommentService.class);  
  
        boolean b1 = cs1 == cs2; ← 두 변수 c1과 c2가 서로 다른 인스턴스를 참조, 결과는 false  
  
        System.out.println(b1);  
    }  
}
```

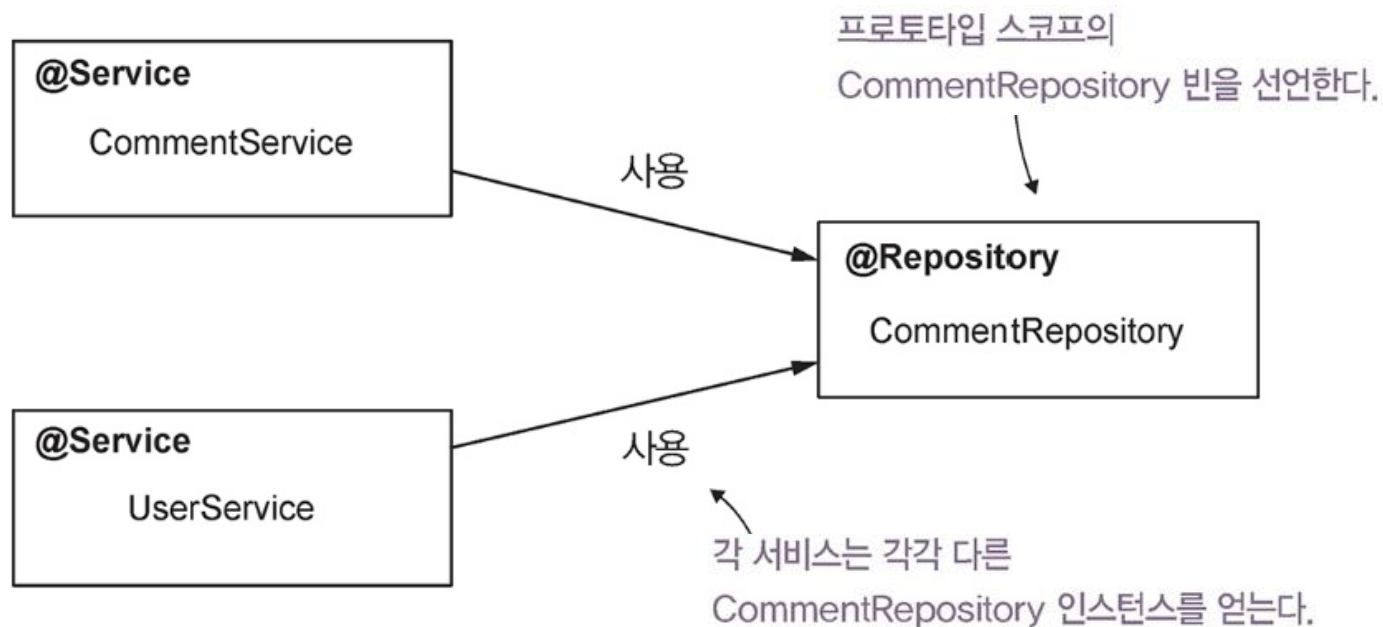
✓ 스테레오타입 애노테이션으로 프로토타입 소코프 빈 선언하기

@Repository

@Scope(BeanDefinition.SCOPE_PROTOTYPE)

```
public class CommentRepository {
```

```
}
```



✓ 스테레오타입 애노테이션으로 프로토타입 소코프 빈 선언하기

@Service

```
public class CommentService {
```

```
    @Autowired
```

```
    private CommentRepository commentRepository;
```

```
    public CommentRepository getCommentRepository() {
```

```
        return commentRepository;
```

```
    }
```

```
}
```

@Configuration

```
@ComponentScan(basePackages = {"services", "repositories"})
```

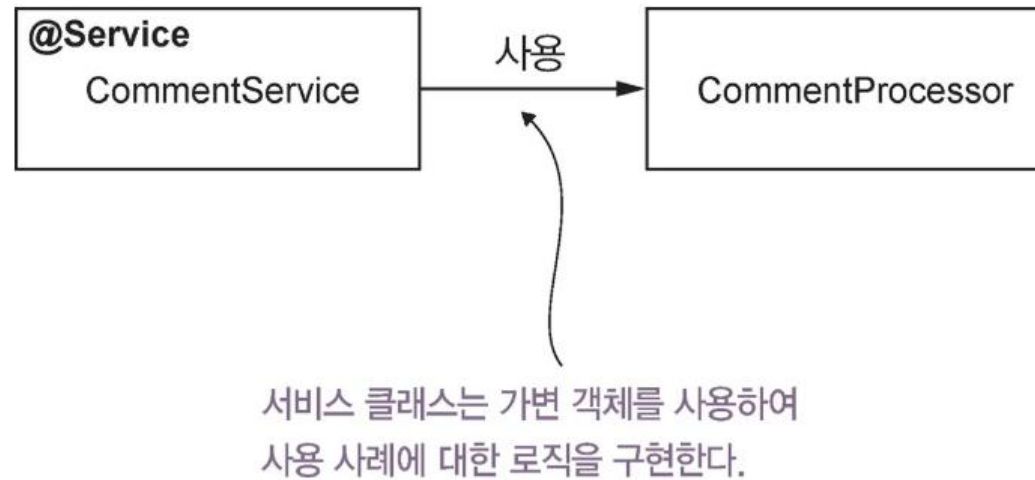
```
public class ProjectConfig {
```

```
}
```

✓ 스테레오타입 애노테이션으로 프로토타입 소코프 빈 선언하기

```
public class Main {  
  
    public static void main(String[] args) {  
        var c = new AnnotationConfigApplicationContext(ProjectConfig.class);  
  
        var s1 = c.getBean(CommentService.class);  
        var s2 = c.getBean(UserService.class);  
  
        boolean b = s1.getCommentRepository() == s2.getCommentRepository();  
  
        System.out.println(b);  
    }  
}
```

✓ 실제 시나리오에서 프로토타입 빈 관리



CommentProcessor빈

```
public class CommentProcessor {  
  
    private Comment comment;  
  
    public void setComment(Comment comment) {  
        this.comment = comment;  
    }  
  
    public Comment getComment() {  
        return this.comment;  
    }  
  
    public void processComment(Comment comment) {  
        // comment의 속성을 변경한다.  
    }  
  
    public void validateComment(Comment comment) {  
        // comment의 속성을 변경한다.  
    }  
  
}
```

CommentService

```
@Service
public class CommentService {

    public void sendComment(Comment c) {
        CommentProcessor p = new CommentProcessor();

        p.setComment(c);
        p.processComment(c);
        p.validateComment(c);

        c = p.getComment();
        // 추가 작업 수행
    }
}
```



CommentProcessor 객체는
스프링 컨텍스트의 빈이 필요하다.
CommentRepository 인스턴스를 얻는
가장 쉬운 방법은 DI를 요청하는 것이다.

CommentProcessor빈

```
@Component
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
public class CommentProcessor {

    @Autowired
    private CommentRepository commentRepository;

    private Comment comment;

    public void setComment(Comment comment) {
        this.comment = comment;
    }

    public Comment getComment() {
        return this.comment;
    }

    public void processComment(Comment comment) {
    }

    public void validateComment(Comment comment) {
    }
}
```

CommentService

○ CommentProcessor를 프로토타입 빈으로 사용하기

```
@Service
public class CommentService {

    @Autowired
    private ApplicationContext context;

    public void sendComment(Comment c) {
        CommentProcessor p = context.getBean(CommentProcessor.class);

        p.setComment(c);
        p.processComment(c);
        p.validateComment(c);

        c = p.getComment();
        // 추가 작업 수행
    }
}
```

CommentService

○ CommentProcessor를 싱글톤으로 주입하기

```
@Service
public class CommentService {

    @Autowired
    private CommentProcessor p;

    public void sendComment(Comment c) {
        p.setComment(c);
        p.processComment(c);
        p.validateComment(c);

        c = p.getComment();
        // 추가 작업 수행
    }
}
```

✓ 싱글톤 및 프로토타입 빈 스코프간의 간단한 비교

싱글톤	프로토타입
- 프레임워크는 이름을 실제 객체 인스턴스와 연관시킨다. - 빈 이름을 참조할 때마다 동일한 객체 인스턴스를 얻게 된다. - 컨텍스트가 로드될 때 또는 처음 참조될 때 인스턴스를 생성하도록 스프링을 구성할 수 있다. - 싱글톤은 스프링의 기본 빈 스코프다. - 싱글톤 빈에 변경 가능한 속성을 포함할 것을 권장하지 않는다.	- 이름은 타입과 연관되어 있다. - 빈 이름을 참조할 때마다 새로운 인스턴스가 생성된다. - 프레임워크는 빈을 참조할 때 항상 프로토타입 스코프에 대한 객체를 생성한다. - 빈을 명시적으로 프로토타입으로 표시해야 한다, - 프로토타입 빈은 변경 가능한 속성을 포함시킬 수 있다.